The Fatigue Curve: A Framework for Quantifying and Mitigating Instruction-Following Decay in Large Language Models

Authors: [Your Name], [Affiliation], [Email] **Status:** Draft v0.1 (experiments pending; all `TBD` fields are filled from notebook outputs) **Code & data:** [GitHub link] · Notebook: `fatigue_curve_pipeline.ipynb`

Abstract

Large language models (LLMs) are routinely deployed as assistants that receive behavioral rules once, such as format, length, tone or policy constraints, and are expected to obey them for the entire conversation. Existing evaluations show that models degrade in multi-turn, long-context and many-instruction settings, but they (i) rarely separate the effect of *turn count* from *token count*, (ii) report point drops rather than a comparable decay model, (iii) mostly test per-turn instructions rather than *persistent* rules, and (iv) seldom compare mitigations at matched token cost. We introduce the **Fatigue Curve**, a reproducible framework that measures the Instruction-Following Rate IFR(t) of persistent, programmatically verifiable rules across a sequence of unrelated tasks, and fits a parametric decay model that yields a decay rate λ, half-life, onset turn and area under the curve. A length-matched control (*flattened history*) and a no-history control (*fresh*) allow us to attribute decay to conversational structure versus context length. We further propose **FAIR** (Fatigue-Aware Instruction Refresh), an adaptive method that re-injects rules only when a violation is detected or a maximum gap is reached, and compare it with periodic reminders, always-remind and sliding-window baselines. Experiments on open-source models run entirely on free-tier hardware show **[TBD: main finding, e.g., "IFR drops from X to Y over N turns (half-life Z turns)"]**, **[TBD: attribution result]**, and **[TBD: FAIR recovers P% of the lost adherence using Q% fewer prompt tokens than always-remind]**. All code, seeds and raw outputs are released.

Keywords: instruction following, multi-turn dialogue, instruction drift, long-context reliability, LLM evaluation, prompt-level mitigation, reproducibility

---

1. Introduction

1.1 Motivation

Production LLM systems (customer-support agents, tutoring systems, coding assistants, compliance bots) specify behavior in a system prompt and then run for dozens or hundreds of turns. Silent failure of those rules has concrete costs: broken output formats that crash downstream parsers, policy violations, tone breaches and unsafe outputs. Practitioners report this decay anecdotally, but there is no standard measure of *how fast* a model's adherence decays, *why* it decays, or *which cheap fix works*.

1.2 Research Gap

Prior work establishes that performance degrades with multi-turn interaction, longer inputs and more simultaneous instructions (Section 2). However, four gaps remain:

1. **G1 (confounded cause).** Turn count and token count rise together. Studies seldom isolate which drives the failure.
2. **G2 (no quantitative model).** Results are reported as before/after drops, not as comparable decay parameters across models and conditions.
3. **G3 (persistent rules).** Most benchmarks issue fresh instructions each turn. The common deployment pattern of *state a rule once, expect it forever* is under-measured.
4. **G4 (unfair mitigation comparison).** Fixes such as reminders, truncation or attention changes are rarely compared head to head at matched token cost, and none adapt to observed model behavior.

1.3 Research Questions and Hypotheses

ID	Research question	Hypothesis	Test
RQ1	Does adherence to persistent rules decay with turn index, and with what parameters?	**H1:** the `turn` slope in `persistent` is significantly negative.	GEE logistic regression; decay fit
RQ2	Is decay driven by turn structure or by context length?	**H2:** `persistent` decays faster than length-matched `flattened`.	Paired comparison of late-phase IFR; GEE interaction
RQ3	Which rule types decay first, and does this depend on model size?	**H3:** format/length rules decay before lexical rules; larger models decay more slowly.	Per-rule curves; cross-model table
RQ4	Can an adaptive refresh recover adherence more cheaply than fixed reminders?	**H4:** FAIR late-phase IFR is not significantly lower than always-remind, with fewer prompt tokens.	Paired Wilcoxon with Holm correction; IFR per 1k tokens

1.4 Contributions

1. A **formal definition and metric suite** for instruction-following decay (IFR(t), λ, half-life, onset, AUC, length-attribution share).
2. A **controlled experimental design** that separates turn structure from token length using a flattened-history control.

3. **FAIR**, a simple adaptive mitigation, benchmarked against three baselines at matched token cost.
4. A **fully reproducible, zero-cost pipeline** (Colab, open models, public data, deterministic decoding).

---

2. Related Work

> Note to author: verify every citation (title, authors, venue, year) and add 2025–2026 follow-up work before submission.

2.1 Instruction-Following Benchmarks

IFEval (Zhou et al., 2023) introduced instructions whose compliance can be checked by code, avoiding judge-model bias; we adopt this principle. FollowBench (Jiang et al., 2023), ComplexBench (Wen et al., 2024) and InFoBench (Qin et al., 2024) add multi-level, composed and decomposed constraints. Harada et al. (2024) and Jaroslawicz et al. (2025) show that success falls as the number of simultaneous instructions grows. These works evaluate a *single* turn and vary the instruction count, not the conversation length.

2.2 Multi-Turn Evaluation

MT-Bench (Zheng et al., 2023), MT-Bench-101 (Bai et al., 2024), MT-Eval (Kwan et al., 2024) and Multi-IF (He et al., 2024) evaluate conversations but typically over few turns and with aggregate scores. Laban et al. (2025) report substantial degradation in multi-turn settings where information arrives incrementally. Our setting differs: the rules are fully specified at the start and the later tasks are unrelated to them.

2.3 Persona and System-Prompt Drift

Li et al. (2024) measure persona drift across dialog turns and relate it to decaying attention on the system prompt, proposing split-softmax. SysBench (Qin et al., 2024) evaluates system-message following. Llama 2's Ghost Attention (Touvron et al., 2023) trains for multi-turn instruction persistence. These are the closest works, but they do not provide a standard decay model, a length-matched control, or a cost-matched comparison of prompt-level mitigations.

2.4 Long-Context Reliability

Liu et al. (2024) show position-dependent retrieval failures ("lost in the middle"). RULER (Hsieh et al., 2024) shows effective context length falls short of the advertised one. Levy et al. (2024) show that added tokens alone can degrade reasoning. Xiao et al. (2024) show the importance of attention sinks. These findings offer a competing explanation for decay (length alone), which our flattened-history control directly tests.

2.5 Positioning

Prior work names the phenomenon (drift), proposes length as a rival cause, and supplies verifiable-evaluation tools. This paper unifies them in a single controlled design and adds a quantitative decay model and a cost-aware, adaptive mitigation.

---

3. Problem Formulation

3.1 Setup

A **sequence** *s* consists of a rule set *R_s* (|R_s| = 2 in our main setting) stated once in the system prompt, and a task stream *x_{s,1}, …, x_{s,T}* of unrelated user requests. At turn *t* the model produces *y_{s,t}*.

Each rule *r* has a verifier *v_r(y) ∈ {0,1}* implemented in code. Turn adherence is

```
a_{s,t} = Π_{r ∈ R_s} v_r(y_{s,t})        (all rules must hold)
```

3.2 Instruction-Following Rate

```
IFR(t) = (1/N) Σ_s a_{s,t}
```

3.3 Fatigue Model

We model the mean curve as exponential relaxation:

```
IFR(t) = a + (b − a) · exp(−λ t),   0 ⩽ a, b ⩽ 1, λ ⩾ 0
```

where *b* is initial adherence, *a* the asymptote, and *λ* the decay rate. Derived metrics:

Metric	Definition	Interpretation
Half-life	ln 2 / λ	Turns until half of the eventual drop has occurred
Onset turn	first *t* where the 3-turn rolling IFR < (mean IFR of turns 1–3) − 0.10	When fatigue becomes visible
AUC	mean of IFR(t) over all turns	Overall reliability
Fatigue gap	IFR_fresh,late − IFR_persistent,late	Total adherence lost to conversation history

"Late" denotes the final third of turns.

3.4 Attribution of Decay to Length vs. Structure

Let $D_total = IFR_fresh,late - IFR_persistent,late$ and $D_len = IFR_fresh,late - IFR_flattened,late$. The **length share** is

```
ρ_len = D_len / D_total
```

If ρ_len ≈ 1, decay is explained by context content/length. If ρ_len ≈ 0, it is specific to conversational turn structure. This quantity is reported only when *D_total* is significantly greater than zero.

3.5 Cost

Prompt-token cost per turn *c(t)* is measured from the tokenizer. Mitigations are compared on late-phase IFR and on **IFR per 1,000 prompt tokens**.

---

4. Method

4.1 Experimental Conditions

Condition	Description	Role
`fresh`	Each task answered independently with the system prompt only	Upper bound / task-difficulty control
`persistent`	Rules once in system prompt; full history accumulates	**Fatigue condition**
`flattened`	Same history as `persistent`, packed into a single user message	**Length control**
`reminder_every_k`	Rules re-appended every $k = 5$ turns	Baseline mitigation
`reminder_every_turn`	Rules re-appended every turn	Costly upper-bound baseline
`window`	System prompt + last $W = 4$ turns	Baseline mitigation (truncation)
`fair`	Proposed adaptive refresh (below)	**Proposed method**

4.2 FAIR: Fatigue-Aware Instruction Refresh

FAIR re-injects the rules only when evidence of slipping appears, rather than on a fixed schedule.

```
Algorithm FAIR
Input: rules R, verifiers {v_r}, max gap K_max
state: last_violated ← False, since ← 0
for t = 1..T:
    if t > 1 and (last_violated or since ⩾ K_max):
        append Reminder(R) to user message x_t ; since ← 0
    else:
        since ← since + 1
    y_t ← LLM(history, x_t)
    last_violated ← ¬ all_r v_r(y_t)
```

Design rationale. Violation-triggered refresh targets turns where the model has demonstrably slipped, while *K_max* provides a safety net for silent decay in settings where the detector is imperfect. In this study the detector is the verifier suite (an optimistic assumption; see Limitations).

4.3 Variants Planned for Ablation

- Detector noise: flip verifier outcomes with probability $p \in \{0, 0.1, 0.2\}$ to simulate an imperfect detector.
- $K_max \in \{5, 10, 20\}$ and $k \in \{3, 5, 10\}$.
- Reminder placement: end of user message vs. start vs. inserted as system message.
- Number of rules $|R| \in \{2, 4, 6\}$.

---

5. Experimental Setup

5.1 Models

Model	Params	Purpose
Qwen2.5-1.5B-Instruct	1.5B	Pilot / small
Qwen2.5-3B-Instruct	3B	Scaling within family
Phi-3.5-mini-instruct (or Llama-3.2-3B-Instruct)	~3.8B / 3B	Cross-family check

All run in fp16 on a free-tier Google Colab T4 GPU (16 GB).

5.2 Tasks

Instructions from the Stanford Alpaca dataset, filtered to those with no auxiliary input and 20–200 characters. Tasks are shuffled with a fixed seed and assigned to (sequence, turn) cells. The **same tasks are used in every condition** (paired design).

5.3 Rules and Verifiers

Four verifiable rules, two sampled per sequence:

Rule	Instruction	Verifier
lowercase	Write entirely in lowercase	`r == r.lower()` and contains a letter
end_tag	End every response with `~done~`	`r.strip().endswith("~done~")`
max_words	Under 60 words	`len(r.split()) < 60`
no_commas	Never use a comma	`"," not in r`

Rule combinations are drawn from the set to avoid contradictory pairs.

5.4 Protocol

- Sequences per model: **N = [TBD, ⩾ 30]**; turns per sequence: **T = [TBD, 30–50]**.
- Decoding: greedy (`do_sample=False`), `max_new_tokens = 150`, fixed seeds.
- Each condition is run to completion and cached to CSV (resume-safe).

5.5 Statistical Analysis

1. **H1:** GEE logistic regression `ok ~ turn`, clustered by sequence, exchangeable correlation.
2. **H2/H4:** GEE with `turn × condition` interaction; paired Wilcoxon signed-rank tests on per-sequence late-phase IFR versus `persistent`, with **Holm-Bonferroni** correction and bootstrap 95% CIs (2,000 resamples).
3. **Curve fitting:** non-linear least squares for the exponential model; **bootstrap over sequences** (⩾ 300 resamples) for λ confidence intervals.
4. **Effect sizes:** reported as absolute IFR differences with CIs, not only p-values.

5.6 Reproducibility

Fixed seeds; deterministic decoding; pinned library versions (see Appendix C); raw responses saved with every verdict; single notebook runs end to end at zero cost.

---

6. Results

> All numbers below are `TBD` and must be copied from notebook outputs. Do not describe an effect that the data do not show.

6.1 Existence and Shape of Fatigue (RQ1)

Table 1. Fatigue-curve parameters (per model, per condition). Source: notebook Cell 9, `summary_metrics.csv`.

Model	Condition	IFR_0 (b)	IFR_∞ (a)	λ [95% CI]	Half-life	Onset	AUC
Qwen2.5-1.5B	fresh	TBD	TBD	TBD	TBD	TBD	TBD
Qwen2.5-1.5B	persistent	TBD	TBD	TBD	TBD	TBD	TBD
Qwen2.5-1.5B	flattened	TBD	TBD	TBD	TBD	TBD	TBD
Qwen2.5-3B	persistent	TBD	TBD	TBD	TBD	TBD	TBD
Phi-3.5-mini	persistent	TBD	TBD	TBD	TBD	TBD	TBD

Figure 1. IFR(t) with bootstrap bands for `fresh`, `persistent`, `flattened` (`fig_fatigue.png`).

Report: GEE slope for `turn` in `persistent` (coefficient, SE, p). **[TBD: supports / does not support H1.]**

6.2 Turn Structure vs. Context Length (RQ2)

Report *D_total*, *D_len*, and ρ_len with bootstrap CI for each model. **[TBD: interpretation: length-driven, structure-driven, or mixed.]**

6.3 Which Rules Decay First (RQ3)

Table 2. Early (first 5 turns) vs. late (last 5 turns) per-rule adherence in `persistent`. Source: notebook Cell 13.

Rule	Early	Late	Δ
lowercase	TBD	TBD	TBD
end_tag	TBD	TBD	TBD
max_words	TBD	TBD	TBD
no_commas	TBD	TBD	TBD

Discuss the model-size trend using the cross-model table (Cell 17).

6.4 Mitigation Comparison (RQ4)

Table 3. Late-phase IFR, token cost and efficiency. Source: notebook Cell 12.

Condition	IFR (all)	IFR (late)	Mean prompt tokens	Reminders/turn	IFR per 1k tokens
persistent	TBD	TBD	TBD	0	TBD
reminder_every_k	TBD	TBD	TBD	TBD	TBD
reminder_every_turn	TBD	TBD	TBD	1.0	TBD
window	TBD	TBD	TBD	0	TBD
fair	TBD	TBD	TBD	TBD	TBD

Table 4. Paired tests versus `persistent` (late-phase IFR, Holm-adjusted). Source: notebook Cell 15.

Condition	Mean Δ	95% CI	p (Holm)
reminder_every_k	TBD	TBD	TBD
reminder_every_turn	TBD	TBD	TBD
window	TBD	TBD	TBD
fair	TBD	TBD	TBD

Figure 2. Mitigation curves (`fig_mitigation.png`).

6.5 Ablations

Detector-noise sensitivity, K_max/k sweeps, reminder placement, rule count. **[TBD]**

---

7. Discussion

1. **Does fatigue exist?** [Interpret H1 outcome with effect size and half-life.]
2. **Why does it occur?** [Interpret ρ_len. If flattened history also fails, length/content interference is implicated. If flattened is near fresh, turn-structure effects such as attention to the system prompt across role-delimited turns are implicated. Frame as a hypothesis, since behavioral experiments cannot prove an internal mechanism.]

3. **Practical guidance.** [State a deployment rule of thumb derived from the half-life and the cost table, e.g., "refresh rules every ≈ X turns or on first violation".]
4. **Which fix to use.** [Position FAIR against always-remind and window in terms of IFR and tokens.]
5. **Model-size effects.** [Report whether larger models show smaller λ or later onset.]

---

8. Limitations

- **Detector assumption.** FAIR uses a code verifier. Real deployments with non-verifiable rules need a learned or LLM-judge detector; the noise ablation (Section 4.3) is a first step, not a replacement.
- **Rule type.** Four surface-level, format-style rules. Semantic or safety rules may behave differently.
- **Model scale.** Small open models (1.5B–4B). Results may not extrapolate to frontier models.
- **Task distribution.** Alpaca-style short instructions; no long documents, tools or code.
- **Single decoding setting.** Greedy only; stochastic decoding may change variance.
- **Mechanism.** The study is behavioral; it does not inspect attention or activations.
- **Conversation realism.** Tasks are unrelated and synthetic rather than naturally flowing dialogues.

---

9. Ethics and Broader Impact

All data are public, contain no personal information, and no human subjects are involved. Better measurement of instruction persistence supports safer deployment, since a rule that silently decays can cause policy violations. A risk is that adaptive refresh could be mis-read as a safety guarantee. It is not, and it should be paired with independent monitoring.

---

10. Conclusion

We introduce the Fatigue Curve, a formal and reproducible way to quantify how instruction adherence decays over a conversation, a control that separates turn structure from context length, and FAIR, an adaptive refresh method compared at matched token cost. **[TBD: one-paragraph summary of the actual findings and their implication.]** Future work includes learned violation detectors, semantic and safety rules, larger models, mechanistic analysis of attention to the system prompt, and training-time remedies.

References

> All entries are from memory and must be verified against arXiv/Google Scholar before submission. Add DOI/arXiv IDs and fix venue details.

1. Zhou, J., et al. (2023). *Instruction-Following Evaluation for Large Language Models* (IFEval). arXiv.
2. Jiang, Y., et al. (2023). *FollowBench: A Multi-level Fine-grained Constraints Following Benchmark for LLMs*.
3. Wen, B., et al. (2024). *Benchmarking Complex Instruction-Following with Multiple Constraints Composition* (ComplexBench).
4. Qin, Y., et al. (2024). *InFoBench: Evaluating Instruction Following Ability in Large Language Models*.
5. Harada, K., et al. (2024). *Curse of Instructions: Large Language Models Cannot Follow Multiple Instructions at Once* (ManyIFEval).
6. Jaroslawicz, D., et al. (2025). *How Many Instructions Can LLMs Follow at Once?* (IFScale).
7. Zheng, L., et al. (2023). *Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena*. NeurIPS Datasets & Benchmarks.
8. Bai, G., et al. (2024). *MT-Bench-101: A Fine-Grained Benchmark for Evaluating LLMs in Multi-Turn Dialogues*. ACL.
9. Kwan, W.-C., et al. (2024). *MT-Eval: A Multi-Turn Capabilities Evaluation Benchmark for LLMs*. EMNLP.
10. He, Y., et al. (2024). *Multi-IF: Benchmarking LLMs on Multi-Turn and Multilingual Instructions Following*.
11. Laban, P., et al. (2025). *LLMs Get Lost in Multi-Turn Conversation*.
12. Li, K., et al. (2024). *Measuring and Controlling Persona Drift in Language Model Dialogs*. COLM.
13. Qin, Y., et al. (2024). *SysBench: Can Large Language Models Follow System Messages?*
14. Touvron, H., et al. (2023). *Llama 2: Open Foundation and Fine-Tuned Chat Models* (Ghost Attention).
15. Liu, N. F., et al. (2024). *Lost in the Middle: How Language Models Use Long Contexts*. TACL.
16. Hsieh, C.-P., et al. (2024). *RULER: What's the Real Context Size of Your Long-Context Language Models?* COLM.
17. Levy, M., Jacoby, A., Goldberg, Y. (2024). *Same Task, More Tokens: the Impact of Input Length on the Reasoning Performance of LLMs*. ACL.
18. Xiao, G., et al. (2024). *Efficient Streaming Language Models with Attention Sinks* (StreamingLLM). ICLR.
19. Wolf, T., et al. (2020). *Transformers: State-of-the-Art Natural Language Processing*. EMNLP Demos.
20. Qwen Team (2024). *Qwen2.5 Technical Report*.

21. Taori, R., et al. (2023). *Stanford Alpaca: An Instruction-following LLaMA Model*. GitHub.

Appendix A. Prompt Templates

System prompt

```
You are a helpful assistant. Follow these rules in EVERY response for the whole
conversation:
- <rule 1 text>
- <rule 2 text>
```

Reminder (appended to the user message)

```
[Reminder of the rules:
- <rule 1 text>
- <rule 2 text>]
```

Flattened-history wrapper

```
Earlier log (context only, do not answer these again):
User: <task 1>
Assistant: <response 1>
...
Now answer this new request:
<current task>
```

Appendix B. Hyperparameters

Parameter	Value
Sequences N	TBD (pilot 10; final ⩾ 30)
Turns T	TBD (pilot 20; final 30–50)
Rules per sequence	2 (ablation: 4, 6)
max_new_tokens	150
Decoding	greedy
Reminder period k	5
Window size W	4 turns
FAIR K_max	10
Bootstrap resamples	300 (curve fit), 2,000 (paired tests)
Seed	42

Appendix C. Reproducibility Checklist

- ☐ Code and notebook in a public repository with a license
- ☐ Seeds and decoding settings fixed and reported
- ☐ Library versions pinned (`transformers`, `torch`, `datasets`, `scipy`, `statsmodels`)
- ☐ Model identifiers and revision hashes recorded
- ☐ Raw responses and per-turn verdicts released (CSV)
- ☐ Compute budget stated (free-tier T4; approximate GPU-hours)
- ☐ All reported numbers regenerated by a single notebook run
- ☐ Negative and null results reported

Appendix D. Qualitative Failure Examples

Insert 3–5 examples from notebook Cell 19, each with sequence rules, turn index and the model output, showing typical violation types (e.g., capitalization creeping back, missing end tag, overly long answers).

Appendix E. Mapping from Paper to Notebook

Paper element	Notebook cell(s)
Rules and verifiers (5.3)	Cell 3
Tasks (5.2)	Cell 4
Conditions and FAIR (4)	Cell 6
Main run (5.4)	Cell 8
Table 1, Fig. 1	Cells 9–10
H1–H2 statistics	Cell 11
Table 3	Cell 12
Table 2	Cell 13
Table 4	Cell 15
Validity checks	Cell 16
Cross-model table	Cell 17
LaTeX tables	Cell 18
Failure examples (App. D)	Cell 19